

Spring Core & Microservices Coding Assignments

1. Spring Core, IOC, Spring Bean, Spring Context

Assignment	Description	Focus
Assignment 1.1: Scope and Lifecycle Management	Create a ShoppingCart bean with the prototype scope and a CheckoutService bean with the singleton scope. The CheckoutService must ensure that every time its getNewCart() method is called, a brand new ShoppingCart instance is returned, effectively demonstrating lookup method injection or use of ObjectFactory.	Bean Scopes, Lookup Injection, ObjectFactory, Singleton/Prototype Interaction
Assignment 1.2: Custom Bean Post Processing	Implement a custom BeanPostProcessor that intercepts the initialization of any bean implementing an interface named Auditable. Before the bean's actual initialization method runs, the post-processor must set a property called auditTrailId to a randomly generated UUID and log a message indicating the bean name and the generated ID.	BeanPostProcessor, Initialization Lifecycle, Custom Logic Injection

2. Spring Boot - Introduction, Auto-Configuration, Runners

Assignment	Description	Focus
Assignment 2.1: Conditional	Create a starter module that provides an EmailService bean. The service should only be auto-configured and available	Custom Auto-Configuration, @ConditionalOnProperty, @Configuration

Assignment	Description	Focus
Auto-Configuration	in the application context if the developer provides a property named mail.server.url in application.properties. If the property is missing, the bean should not be present.	
Assignment 2.2: Context Initialization and Configuration Loading	Implement both an ApplicationRunner and a CommandLineRunner. The ApplicationRunner must access command-line arguments, and the CommandLineRunner must access a list of properties defined in application.yml using the @Value annotation. Log the accessed values to demonstrate execution after context setup.	ApplicationRunner, CommandLineRunner, @Value, Configuration Loading

3. Spring Data JPA

Assignment	Description	Focus
Assignment 3.1: Repository Method Derivation and Paging	Create a JPA entity Product and a JpaRepository. Implement a service method that accepts a product category name and a Pageable object (page number, size, and sort direction). Write a Derived Query Method in the repository named findByCategoryAndPriceGreaterThan that returns a paginated list of products.	Derived Query Methods, JpaRepository, Pageable Interface

Assignment	Description	Focus
Assignment 3.2: Transaction Management and Custom Queries	Implement a service method annotated with @Transactional that performs two database operations: 1) Decrements the stock of a product using a custom JPQL query defined with @Query, and 2) Updates the product's last modified date using a standard save() call. Introduce a runtime exception between the two operations and verify that both operations are rolled back.	@Transactional, JPQL @Query, Rollback Mechanism

4. Spring Data - No SQL

Assignment	Description	Focus
Assignment 4.1: MongoDB Aggregation and Repository Setup	Configure a Spring Boot application to use Spring Data MongoDB. Create a Transaction document. Implement a repository method that uses the MongoDB Aggregation Framework via the Aggregation class to calculate the total transaction amount grouped by user ID and then sort the results in descending order.	Spring Data MongoDB, @Document, Aggregation Framework
Assignment 4.2: Redis Caching with TTL	Integrate Spring Data Redis as a caching layer. Annotate a service method, getUserDataById(String userId), with @Cacheable. Configure the cache to store user data for a Time-To-Live (TTL) of 60 seconds , demonstrating that subsequent calls within the time limit retrieve the data from the cache.	Spring Data Redis, @Cacheable, Cache TTL Configuration

5. REST API using Spring Boot, Global Exception Handling

Assignment	Description	Focus
Assignment 5.1: Resource Validation and Custom Response	Create a REST controller with a POST endpoint that accepts a DTO. Use JSR-303 annotations (e.g., @Valid, @NotNull) for validation. If validation fails, implement a Global Exception Handler using @ControllerAdvice and @ExceptionHandler to catch MethodArgumentNotValidException and return a custom JSON error response with an HTTP status code of 400 Bad Request , listing all validation errors.	@RestController, JSR-303 Validation, @ControllerAdvice, Custom Error Payload
Assignment 5.2: RESTful API Design and Status Codes	Create a simple ItemController with methods for creating (POST), retrieving by ID (GET), and deleting (DELETE) an item. Ensure that the POST method returns HTTP status 201 Created and includes the location URI of the new resource in the response header. The DELETE method should return 204 No Content .	RESTful principles, HTTP Status Codes (201, 204), ResponseEntity

6. Unit Testing & Swagger UI

Assignment	Description	Focus
Assignment 6.1: MockMvc Controller Testing	Write an isolated unit test for the ItemController from Assignment 5.2 using @WebMvcTest and MockMvc . Use Mockito to mock the underlying service layer. Write tests to verify: 1) a successful POST request returns the correct JSON body and 2) a GET request for a non-	@WebMvcTest, MockMvc, Mockito Mocks, JSON Assertions

Assignment	Description	Focus
	existent ID returns HTTP status 404 Not Found .	
Assignment 6.2: Open API Documentation with Springdoc	Integrate Springdoc OpenAPI (Swagger) into the Spring Boot application. Annotate the <code>ItemController</code> and its methods using <code>@Tag</code> , <code>@Operation</code> , and <code>@ApiResponse</code> to provide detailed information about the endpoints, request schemas, and possible HTTP response codes (e.g., 200, 201, 400). Verify the documentation is correctly displayed at the Swagger UI URL.	Springdoc OpenAPI, <code>@Tag</code> , <code>@Operation</code> , API Documentation

7. Asynchronous Programming, Reactive REST API

Assignment	Description	Focus
Assignment 7.1: Asynchronous Service with Thread Pool	Configure an application-specific TaskExecutor (custom thread pool). Implement a service method using the @Async annotation that simulates a long-running process (e.g., <code>Thread.sleep(5000)</code>). The method must return a CompletableFuture<String> . Call this method from a controller and demonstrate that the controller thread does not block while waiting for the result.	<code>@EnableAsync</code> , <code>TaskExecutor</code> , <code>@Async</code> , <code>CompletableFuture</code>
Assignment 7.2: Reactive	Create a Spring WebFlux application. Implement a GET endpoint that	Spring WebFlux, <code>Flux<T></code> , Server-Sent

Assignment	Description	Focus
Stream Processing with WebFlux	returns a Flux<StockPrice> . The endpoint should simulate a real-time stock feed by emitting a new StockPrice object every 1 second, streaming the data back to the client as Server-Sent Events (SSE) .	Events (SSE), Non-blocking I/O

8. Authentication and Authorization using Basic Auth and JWT, Spring Security

Assignment	Description	Focus
Assignment 8.1: Basic Auth and Role-Based Authorization	Configure Spring Security to use Basic Authentication for all requests. Create two REST endpoints: /api/admin and /api/user. Implement role-based authorization to ensure that only users with the ADMIN role can access /api/admin, and users with either ADMIN or USER roles can access /api/user.	Spring Security Configuration, Basic Auth, Role-based Authorization
Assignment 8.2: JWT Generation and Validation	Implement a custom JWT login endpoint that, upon successful basic authentication, generates a JSON Web Token (JWT) containing the user's roles. Subsequent requests to a secured endpoint should require the client to present this JWT in the Authorization: Bearer header, which the security filter chain must validate.	JWT Generation, Stateless Security, Custom Filter Chain, Authorization: Bearer

9. Microservices Decomposition & Inter-service Communication using Open Feign

Assignment	Description	Focus
Assignment 9.1: Feign Client with Service Discovery	Create two services: OrderService and InventoryService. Configure both services to register with a Service Discovery Server (e.g., Eureka - see section 10). In the OrderService, define a Feign Client interface (annotated with @FeignClient) to call the InventoryService's endpoint for checking stock. Demonstrate the successful inter-service call using the service name instead of a fixed URL.	Open Feign, @FeignClient, Service Name Resolution
Assignment 9.2: Feign Error Handling	Enhance the Feign Client interface from Assignment 9.1 by implementing a custom FeignErrorDecoder. When the InventoryService returns a 404 Not Found (e.g., product not found), the decoder should translate this into a specific custom Java exception (e.g., InventoryNotFoundException) that the calling OrderService can handle gracefully.	Feign Client Error Handling, ErrorDecoder, Custom Exceptions

10. Eureka Discovery Service, API Gateway using Spring Cloud API Gateway

Assignment	Description	Focus
Assignment 10.1: Eureka Server and Client Registration	Set up a central Eureka Server using @EnableEurekaServer. Create two microservices (Service-A and Service-B) and configure them as Eureka Clients using @EnableDiscoveryClient to register with the server. Verify that both services appear in the Eureka dashboard.	Eureka Server, @EnableEurekaServer, Eureka Client, @EnableDiscoveryClient

Assignment	Description	Focus
Assignment 10.2: API Gateway Routing and Filtering	Set up a Spring Cloud API Gateway application. Configure routing rules in application.yml to route all requests with the path prefix /api/serviceA/** to Service-A using Eureka discovery (i.e., lb://SERVICE-A). Add a custom filter to the route that inserts a specific custom header (e.g., X-Request-Source: Gateway) into the request before forwarding it to Service-A.	Spring Cloud Gateway, Dynamic Routing, Custom Gateway Filters

11. Client Side Load Balancing, Config Service

Assignment	Description	Focus
Assignment 11.1: Load Balancing with DiscoveryClient	Run two instances of Service-A (on different ports). Configure Service-B to use a RestTemplate annotated with @LoadBalanced for calling Service-A. Demonstrate that repeated calls from Service-B are correctly distributed between the two instances of Service-A by logging the instance port in Service-A.	Client-Side Load Balancing (@LoadBalanced), RestTemplate, Service Discovery
Assignment 11.2: Centralized Configuration and Refresh Scope	Set up a Spring Cloud Config Server backed by a local Git repository. Create a property app.greeting.message in the configuration. In a client microservice, inject this property using @Value and use the	Spring Cloud Config Server, @RefreshScope, Runtime Configuration Update

Assignment	Description	Focus
	@RefreshScope annotation on the controller. Change the message in the Git repository and demonstrate that the running microservice updates its configuration when the /actuator/refresh endpoint is triggered.	

12. Fault Tolerance using Resilience 4j, Circuit Breaker

Assignment	Description	Focus
Assignment 12.1: Circuit Breaker with Fallback Method	Configure Resilience4j in a client service making a remote call (e.g., using a Feign client). Wrap the remote call method with the @CircuitBreaker annotation. Implement a fallback method that returns a default response when the remote service is unavailable. Test by stopping the remote service and verifying the fallback is executed.	Resilience4j @CircuitBreaker, Fallback Method, Failure Handling
Assignment 12.2: Rate Limiter Implementation	Implement the Rate Limiter pattern using Resilience4j on a public REST endpoint (e.g., /api/high-traffic). Configure the limiter to allow a maximum of 5 requests per second . Use a client utility (e.g., Jmeter or a simple loop) to send 10 requests rapidly and verify that the 6th and	Resilience4j @RateLimiter, Concurrency Control, Testing Throttling

Assignment	Description	Focus
	subsequent requests are blocked or return a rate-limiting exception.	

13. Centralized Logging and Distributed Tracing

Assignment	Description	Focus
Assignment 13.1: Log Correlation with MDC	Configure a custom logging pattern (using Logback or Log4j2) that includes the Trace ID and Span ID generated by Spring Cloud Sleuth (or Micrometer Tracing). Ensure that these IDs are automatically added to the logging context using MDC (Mapped Diagnostic Context) . Call two services sequentially and verify that a single request generates logs with the same Trace ID across both services.	Spring Cloud Sleuth/Micrometer Tracing, MDC, Log Pattern Configuration
Assignment 13.2: Custom Span Creation	In a client service that calls a database, use the tracing API (e.g., Tracer bean in Micrometer) to manually create a custom span around the database access logic. Name the span clearly (e.g., "db-query-execution"). Verify that this custom span appears in the distributed tracing visualization tool (assuming one is set up, like Zipkin or Jaeger).	Distributed Tracing, Manual Span Creation, Tracer Bean

14. Kafka Messaging, Work Queues

Assignment	Description	Focus
Assignment 14.1: Asynchronous Message Production	Configure a Spring Boot application as a Kafka producer. Implement a service method that sends a complex JSON object (e.g., an Order object) to a Kafka topic named orders-queue. Use the asynchronous capabilities of the Kafka template and add a callback that logs whether the message was successfully published, including the partition and offset number.	Kafka Producer, KafkaTemplate, Asynchronous Send, Producer Callback
Assignment 14.2: Work Queue with Consumer Groups	Configure two separate Spring Boot applications (Worker-1 and Worker-2) to act as consumers for the work-tasks Kafka topic. Both consumers must belong to the same Consumer Group ID . Send 10 messages to the topic and verify that each message is processed by only one of the worker instances, successfully demonstrating the Work Queue pattern.	Kafka Consumer, @KafkaListener, Consumer Group, Work Queue Pattern